Manual do Solver

SchrodingerSolver

Elmer FEM 9.0 — SchrodingerSolve

Módulo: `SchrodingerSolve`

Sub-rotina: `SchrodingerSolver`

Arquivo-fonte: `01_simulacoes/schrodinger_P1_2/fonte/SchrodingerSolver.F90` (460 linhas)

Linguagem: Fortran 90

Versão do Elmer: 9.0

> *Este manual descreve a **implementação e o uso** do solver. Para a física e as keywords, ver `Manual_Modelo_Schrodinger.md`.*

Sumário

1. Visão geral[1]
2. Estrutura do código[2]
3. Interface com o Elmer[3]
4. Compilação e instalação[4]
5. Uso — SIF mínimo[5]
6. Saída e interpretação do log[6]
7. Exportação de matrizes[7]
8. Peculiaridades de implementação[8]
9. Diagnóstico de erros[9]
10. Extensão do solver[10]

---

1. Visão geral

O `SchrodingerSolver` é um **solver de usuário** do Elmer (não faz parte da distribuição oficial). Ele resolve o autoproblema generalizado `A ψ = E M ψ` para 1D, 2D e 3D.

[1] \{}#1-visão-geral
[2] \{}#2-estrutura-do-código
[3] \{}#3-interface-com-o-elmer
[4] \{}#4-compilação-e-instalação
[5] \{}#5-uso--sif-mínimo
[6] \{}#6-saída-e-interpretação-do-log
[7] \{}#7-exportação-de-matrizes
[8] \{}#8-peculiaridades-de-implementação
[9] \{}#9-diagnóstico-de-erros
[10] \{}#10-extensão-do-solver

O que ele faz de diferente dos solvers nativos

Aspecto	Solvers nativos do Elmer	`SchrodingerSolver`
Montagem	matriz **esparsa global** via `DefaultUpdateEquations`	matrizes **densas locais** (`ALL`
Autossolver	`EigenSolve.F90` (ARPACK/iterativo)	**`DSYGV`** (LAPACK, denso)
Paralelismo	MPI nativo	**nenhum** (ver §8.4)
Dependência	infraestrutura do Elmer	apenas `DefUtils` + LAPACK

Motivo da escolha: o autossolver ARPACK interno do Elmer apresentou problemas nesta versão/plataforma, e o `DSYGV` independente foi a rota que se mostrou confiável e verificável. O custo é não ter paralelismo nem montagem esparsa — compensado pela exportação de matrizes (§7).

Escopo verificado

		Dimensão	Status	Verificação
1D	✅			erro 3.3e−04; concordância 4.2e−11 com solver Python independen
2D	✅			convergência $O(h^2)$ ordem 1.99
3D	✅			degenerescência tripla correta
$V \neq 0$ 1D/2D/3D	✅			harmônico, Morse, poço duplo, senoidal

---

2. Estrutura do código

```
SchrodingerSolver.F90
├── SUBROUTINE SchrodingerSolver( Model, Solver, dt, Transient )   [linha 50]
│   ├── 1) leitura de parâmetros        (Eigen System Values, Apply Dirichlet, ...)
│   ├── 2) detecção dos nós de contorno (geométrica, independente de BC)
│   ├── 3) leitura do potencial V       (Potential / Potential X,Y,Z / V=0)
│   ├── 4) montagem: laço sobre elementos
│   │       ├── quadratura de Gauss (Integ)
│   │       ├── Stiff    = ∫ ∇Nᵢ·∇Nⱼ
│   │       ├── Mass     = ∫ NᵢNⱼ
│   │       └── PotMass  = ∫ Nᵢ V Nⱼ
│   ├── 5) A = K + Mᵃ ;  B = M
│   ├── 6) eliminação dos GDL fixos -> Af, Bf
│   ├── 7) DSYGV(Af, Bf) -> autovalores e autovetores
│   ├── 8) gravação dos autovalores; modo 1 no campo de saída
│   ├── 9) exportação opcional de A, M, fixed
│   └── 10) desalocação
└── SUBROUTINE ExportaMatriz( nome, Mat, n, arquivo )              [linha 432]
```

Variáveis principais

Variável	Tipo	Papel
`A, B`	`REAL, ALLOCATABLE (:,:)`	matrizes globais densas (operador e massa)
`Af, Bf`	`REAL, ALLOCATABLE (:,:)`	versão reduzida (só GDL livres)
`Stiff, Mass, PotMass`	`REAL, ALLOCATABLE (:,:)`	matrizes elementares
`EigenVals`	`REAL, ALLOCATABLE (:)`	autovalores (as energias)
`xfree`	`REAL, ALLOCATABLE (:,:)`	autovetores, coluna `i` = modo `i`
`Fixed`	`LOGICAL, ALLOCATABLE (:)`	`.TRUE.` para nós com $\psi = 0$
`IdxFree`	`INTEGER, ALLOCATABLE (:)`	índices (globais) dos GDL livres
`Vnodal`	`REAL, ALLOCATABLE (:)`	potencial avaliado em cada nó

---

3. Interface com o Elmer

Assinatura obrigatória

```
SUBROUTINE SchrodingerSolver( Model, Solver, dt, Transient )
  USE DefUtils
  TYPE(Solver_t) :: Solver
  TYPE(Model_t)  :: Model
  REAL(KIND=dp)  :: dt
  LOGICAL        :: Transient
```

A assinatura segue o padrão do Elmer para solvers de usuário. `USE DefUtils` dá acesso a `ListGetInteger`, `ListGetLogical`, `ListGetString`, `ListGetReal`, `ListCheckPresent`, `CoordinateSystemDimension`, `GetElementNodes`, `GaussPoints`, `Info`, `Message`, `I2S`, `MAX_NAME_LEN`.

> ***`USE DefUtils` exporta uma sub-rotina chamada `Info`.*** *Se o código declarar uma variável local `INFO` (como o argumento de retorno do LAPACK), há colisão de nomes e a compilação falha em cascata.* ***Use outro nome*** *— neste solver a variável do LAPACK chama-se* ***`info_lap`****.*

Fluxo de dados

```
SIF  ⟶  Model % BodyForces(1) % Values  ⟶  ListGetReal('Potential')
     ⟶  Solver % Values                 ⟶  ListGetInteger/Logical/String
     ⟶  Model % NumberOfBulkElements, Element % NodeIndexes, Mesh % Nodes
     ⟵  Solver % Variable % Values      ⟵  autofunção (modo 1)
```

Dimensão do problema

```
dim = CoordinateSystemDimension()
```

`dim` vem do `Coordinate System` no SIF, **não** da malha. Por isso é obrigatório que os dois coincidam:

`Coordinate System`	Malha	Resultado
`Cartesian 2D`	2D	✅
`Cartesian 2D`	3D	❌ `ERROR:: ElementMetric: Degenerate 3D element`

---

4. Compilação e instalação

4.1 Pré-requisito: Elmer 9.0 compilado

O solver precisa dos **módulos `.mod`** e da **`libelmersolver.so`** do Elmer.
Neste projeto existe uma build GNU com MUMPS:

```
~/elmer_build/build_mumps/          # árvore de build
~/elmer_build/install_mumps/        # instalação
```

A receita completa da build está em `01_simulacoes/build_elmer/LEIA-ME.md`.

4.2 Compilar o solver

No headnode (compilação é permitida; simulação não):

```
source /etc/profile.d/modules.sh
module load gnu12 openmpi4 openblas
export LC_ALL=C

BLD=$HOME/elmer_build/build_mumps
INST=$HOME/elmer_build/install_mumps

# opcional: headers C empacotados (os nós computacionais não têm /usr/include)
export CPATH="$HOME/sysinclude/usr/include${CPATH:+:$CPATH}"

cd ~/schrodinger_P1_2/build_mumps
mpif90 -c SchrodingerSolver.F90 -o SchrodingerSolver.o \
   -fallow-argument-mismatch -DCONTIG= -DHAVE_EXECUTECOMMANDLINE \
   -DUSE_ISO_C_BINDINGS -O2 -fPIC -I$BLD/fmodules

mpif90 -shared SchrodingerSolver.o -o SchrodingerSolver.so \
   -L$BLD/fem/src -lelmersolver

cp -f SchrodingerSolver.so $INST/share/elmersolver/lib/
```

4.3 Flags explicadas

Flag	Por quê
`-fallow-argument-mismatch`	gfortran 12 é estrito; a API do Elmer tem interfaces implícitas
`-DCONTIG=`	o Elmer usa esse macro para `CONTIGUOUS`; vazio = desabilitado
`-DHAVE_EXECUTECOMMANDLINE`	habilita execução de linha de comando na lib
`-DUSE_ISO_C_BINDINGS`	liga os bindings C do Elmer
`-fPIC`	**obrigatório** para gerar `.so`
`-I$BLD/fmodules`	onde estão os `.mod` do Elmer

4.4 Ambiente de execução

```
export ELMER_HOME=$HOME/elmer_build/install_mumps
export ELMER_SOLVER_HOME=$ELMER_HOME/share/elmersolver
export LD_LIBRARY_PATH=$HOME/elmer_build/install_mumps/lib/elmersolver:
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/opt/ohpc/pub/libs/gnu12/openmpi4/mumps/5.2.1/lib
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/opt/ohpc/pub/libs/gnu12/openmpi4/scalapack/2.2.0/lib
export ES=$ELMER_HOME/bin/ElmerSolver
```

4.5 Lista de verificação

```
# 1) o .so está instalado?
ls -la $ELMER_SOLVER_HOME/lib/SchrodingerSolver.so

# 2) o Elmer carrega o módulo?
$ES caso.sif 2>&1 | grep -i "SchrodingerSolver"
# esperado: Loading user function library: [SchrodingerSolver]...[SchrodingerSolver_Init0]
```

---

5. Uso — SIF mínimo

```
Header
  Mesh DB "." "malha"
End

Simulation
  Coordinate System = Cartesian 1D
  Simulation Type = Steady State
  Steady State Max Iterations = 1
  Max Output Level = 5
End

Body 1
  Equation = 1
  Material = 1
End

Material 1
```

```
  Density = 1.0
End

Equation 1
  Active Solvers(1) = 1
End

Solver 1
  Equation = "Schrodinger"
  Procedure = "SchrodingerSolver" "SchrodingerSolver"
  Variable = "psi"
  Variable DOFs = 1
  Eigen System Values = 5
  Apply Dirichlet = Logical True
  Linear System Solver = Direct
  Linear System Direct Method = "umfpack"
End
```

Com `Potential` (1D) acrescente ao `Body 1` a referência `Body Force = 1` e a seção:

```
Body Force 1
  Potential = Variable Coordinate 1
    Real MATC "1.0*tx*tx"
End
```

Execução

```
$ES caso.sif > run.log 2>&1
grep "E(" run.log
```

---

6. Saída e interpretação do log

Cabeçalho

```
SchrodingerSolver: ======================================
SchrodingerSolver: Solver de Schrodinger (FEM) iniciado
SchrodingerSolver: Numero de autovalores pedidos: 5
SchrodingerSolver: Dimensao do problema: 2
SchrodingerSolver: ======================================
```

Contorno e potencial

```
SchrodingerSolver: Nos de contorno (Dirichlet psi=0): 160
SchrodingerSolver: V(x) lido como campo nodal de Body Force.
SchrodingerSolver:   V_min =   0.000000E+00   V_max =   5.000000E+01
```

Linha	Significado
`Nos de contorno`	quantos nós receberam `ψ = 0` (detecção geométrica)
`V(x) lido como campo nodal de Body Force`	o potencial veio de `Potential`
`V(x,y,z) construido de Potential X/Y/Z`	veio das componentes
`Sem potencial (V = 0)`	**nenhum** potencial foi encontrado
`V_min` / `V_max`	**verificação rápida** de que o potencial está correto

> ***Sempre confira `V_min/V_max`.*** *Este solver teve um defeito em que o potencial era lido de uma seção inválida e `V = 0` era usado* ***em silêncio*** *— ver `docs/VALIDACAO_POTENCIAL.md`. A mensagem `Sem potencial (V = 0)` foi adicionada justamente para tornar isso visível.*

Resultado

```
SchrodingerSolver: DOFs livres (apos eliminacao): 1521
SchrodingerSolver: --- AUTOVALORES (energias) ---
SchrodingerSolver:  E(   1) =   1.028585384356E+00
SchrodingerSolver:  E(   2) =   2.043966018888E+00
SchrodingerSolver:  E(   3) =   2.043966018888E+00
SchrodingerSolver: Autoproblema resolvido (DSYGV).
SchrodingerSolver: Concluido.
```

Se `DSYGV retornou info = <valor>` aparecer, houve problema na diagonalização. `info > 0` indica falha de convergência do algoritmo QR; `info < 0` indica argumento inválido.

Sensibilidade a `Max Output Level`

O solver reporta em **`Level=4`**. Com `Max Output Level = 3` **toda** a saída acima fica suprimida — o solver parece não fazer nada.

---

7. Exportação de matrizes

7.1 Ativação

```
Solver 1
  ...
  Exportar Matrizes = Logical True
  Exportar Prefixo = String "schrod"
End
```

7.2 Arquivos gerados

`schrod_A.dat` — operador `A = K + Mᵃ`:

```
% A  n=441  nnz=3721
441 441 3721
0 0   5.9583333333333357E+00
0 1  -2.0833333333333330E+00
...
```

Formato: `n n nnz` e depois `i j valor`, com `i`, `j` em base **0**.

`schrod_M.dat` — matriz de massa, mesmo formato.

`schrod_fixed.dat` — GDL de Dirichlet:

```
80
0
1
...
```

Primeira linha = contagem; seguintes = índices (base 0).

7.3 Resolver o que foi exportado

```
python3 01_simulacoes/schrodinger_P1_2/scripts/resolve_exportado.py schrod 5
```

Saída verificada **idêntica** ao `DSYGV` interno:

	E(1)	E(2)
Elmer (`DSYGV`)	1.113644656498	2.172208383659
Exportado + ARPACK esparso	1.113644656498	2.172208383659

Com memória **52× menor**. Para malhas grandes é o caminho recomendado.

7.4 Por que exportar

O `DSYGV` é denso: memória `O(n²)`, custo `O(n³)`. Medições:

GDL	denso	esparso	ganho
6 561	41,06 s	0,05 s	821×
14 641	491,60 s	0,12 s	4 097×
39 601	inviável (37,6 GB)	0,41 s	—

8. Peculiaridades de implementação

8.1 `ListGetReal` é função e retorna `F(N)`

Diferente de `ListGetConstReal` (escalar), para ler **campo nodal** é preciso:

```
Vtmp = ListGetReal( BFList, 'Potential', nNodes, NodeIdx, Found )
```

- é **função** (não sub-rotina): `CALL ListGetReal(...)` **não compila**
- o resultado é um **array fixo** `REAL :: F(N)`, não ponteiro — portanto não

cabe `Vtmp => ListGetReal(...)`

- `GetRealValues` (variante) depende de `GetCurrentElement` e **não serve**

fora do laço de montagem

8.2 Colisão de nomes em Fortran

Fortran é **case-insensitive**. Neste projeto houve duas colisões reais:

Nome pretendido	Colidiu com	Solução
`BF` (Body Force)	`Bf` (matriz reduzida)	renomeado para **`BFList`**
`INFO` (LAPACK)	`Info` (sub-rotina de `DefUtils`)	renomeado para **`info_lap`**

8.3 Eliminação de Dirichlet, não penalização

GDL fixos são **eliminados** da matriz (`Af`, `Bf` com apenas as linhas/colunas livres) e não substituídos por `diagonal = 1`. A penalização introduz autovalores espúrios exatamente iguais a 1 — observado empiricamente (`0.829, 1.0, 1.0, 1.0, 1.0` em 3D).

8.4 Não é MPI-aware

O solver monta matrizes **densas locais**. Em execução com `mpirun -np ≥ 2`:

- cada processo enxerga apenas sua **partição** da malha

(`Model % NumberOfNodes` e `Element % NodeIndexes` são **locais**)

- o resultado **não crasha mas está errado**: `E(1) = 3.05` em vez de `1.03`

Além disso, o Elmer 9.0 **não consegue particionar malha** nesta plataforma (colisão em `MeshPartition` → `i2s`), e a ausência de `malha/partitioning.N/` provoca `SIGSEGV` em `ModelDescription.F90:2895`

(NULL deref).

Conclusão: não use `mpirun` com este solver. Para paralelismo, exporte as matrizes (§7) e resolva com PETSc/SLEPc. Análise completa em `docs/PARALELISMO.md`.

8.5 Saída VTU limitada ao modo 1

O campo `psi` recebe **apenas o autovetor do modo 1**. Tentativas de publicar todos os modos com `VariableAdd` (variáveis `psi_1`…`psi_N`) causaram **corrupção de heap** (exit 134/139) e foram revertidas.

Alternativa: resolver as matrizes exportadas com o `resolve_exportado.py` e obter todos os modos.

---

9. Diagnóstico de erros

9.1 Tabela de sintomas

Sintoma	Causa provável	Solução
`cannot open shared object file: SchrodingerSolver.so`	`ELMER_SOLVER_HOME` errado, ou `.so` não ins
`Unknown specifier ... property name:[potential]`	`Potential` colocado em `Material`
`MATC ERROR: Index out of bounds`	`tx(1)` com só uma variável declarada
`Can't find independent variable`	variáveis separadas por **espaço**
`ElementMetric: Degenerate 3D element`	`Coordinate System` ≠ dimensão da malha
Nenhuma saída do solver	`Max Output Level < 4`
`Sem potencial (V = 0)` inesperado	potencial não lido
`DSYGV retornou info = N`	falha na diagonalização
autovalores `1.0` repetidos	penalização de Dirichlet
resultado errado com `mpirun`	solver não é MPI-aware

9.2 Erros de compilação

Erro	Causa	Solução
`Symbol 'bf' already has basic type of DERIVED`	colisão `BF`/`Bf`
`'listgetreal' has a type, not consistent with the CALL`	usado como sub-rotina
`Target expression in pointer assignment must deliver a pointer`	`=>` com função
`'Info' at (1) has a type...`	`INFO` local colide
`Can't open included file 'mpif.h'`	compilador errado no CMake

9.3 Reproduzir a suíte de verificação

```
# 5 casos 1D com solução analítica; esperado: 5 passaram, 0 falharam
qsub -v CASO=... pbs/roda_caso_elmer.pbs
# ou a suíte completa:
bash ~/schrodinger_P1_2/regressao.sh
```

---

10. Extensão do solver

10.1 Torná-lo MPI-aware (o caminho "correto")

Exigiria reescrever a montagem usando a infraestrutura nativa do Elmer:

```
CALL DefaultInitialize()
DO t = 1, Solver % NumberOfActiveElements      ! NUNCA Model % NumberOfBulkElements
  Element => GetActiveElement(t)
  CALL DefaultUpdateEquations( STIFF, FORCE )  ! monta K
  CALL DefaultUpdateMass( MASS )               ! monta M
END DO
CALL DefaultFinishAssembly()
CALL DefaultDirichletBCs()
CALL DefaultSolve()
```

Pontos críticos:

- a matriz global é criada pelo **driver**, não pelo solver

(`ElmerSolver.F90:803`); não alocar `A(nDof,nDof)`

- `Solver % NumberOfActiveElements` ≠ `Model % NumberOfBulkElements`

(o laço atual inclui halo e conta em dobro)

- `Linear System Symmetric = Logical True` é **obrigatório** para que

`CRS_ElimSymmDirichlet` (`SolverUtils.F90:8711`) rode e

`CRSMatrix.F90:1285` zere também as colunas da massa

- template ideal já existe no Elmer: `modules/DFTSolver.F90` →

`WavefunctionSolver` (linhas 218–505), que resolve $(-\frac{1}{2}\nabla^2+V+Vxc)\psi=E\psi$

Esforço estimado: 3–5 dias. **Risco:** alto — o autossolver ARPACK nativo, em paralelo, **converge mas devolve autovalores errados** (testado num caso SPD com analítico conhecido: resultados negativos).

10.2 Publicar todos os modos no VTU

Requer corrigir a corrupção de heap em `VariableAdd` (§8.5). A causa provável é a **propriedade do array de permutação** — o Elmer guarda o ponteiro, e reusar

o mesmo `PermModos` para várias variáveis pode invalidar referências.
Alternativas: alocar um `Perm` por modo, ou escrever arquivos separados por modo.

10.3 Condição de contorno de radiação

Para barreiras e estados ressonantes seria necessário implementar a condição de Sommerfeld ou *complex scaling*. Referência metodológica: `00_planejamento/referencias/` — `qsolver` (CPC 329 (2026) 110393), que usa mapeamento analítico do domínio infinito + colocação polinomial.

Apêndice — referência rápida

```
# COMPILAR (headnode)
BLD=$HOME/elmer_build/build_mumps; INST=$HOME/elmer_build/install_mumps
cd ~/schrodinger_P1_2/build_mumps
mpif90 -c SchrodingerSolver.F90 -o SchrodingerSolver.o -fallow-argument-mismatch \
  -DCONTIG= -DHAVE_EXECUTECOMMANDLINE -DUSE_ISO_C_BINDINGS -O2 -fPIC -I$BLD/fmodules
mpif90 -shared SchrodingerSolver.o -o SchrodingerSolver.so -L$BLD/fem/src -lelmersolver
cp -f SchrodingerSolver.so $INST/share/elmersolver/lib/

# RODAR (nós computacionais!)
export ELMER_HOME=$HOME/elmer_build/install_mumps
export ELMER_SOLVER_HOME=$ELMER_HOME/share/elmersolver
export LD_LIBRARY_PATH=$ELMER_HOME/lib/elmersolver:/opt/ohpc/pub/libs/gnu12/openmpi4/mumps/5.2.1/lib:/
cd ~/P1_2_casos/casos/<caso>
python3 gerar_malha.py
$ELMER_HOME/bin/ElmerSolver caso.sif > run.log 2>&1
grep "E(" run.log
paraview malha/run_t0001.vtu
```

Última atualização: 08/10/2026